변수 할당

개요

변수 할당은 R1CS 제약 시스템에서 각 변수에 구체적인 값을 할당하는 과정입니다. 증인(witness) 생성과 검증의 핵심 구성 요소입니다.

변수 분류

Primary Input (공개 입력)

- **정의**: 검증자와 증명자 모두 알고 있는 공개 값들
- **용도**: 명제의 공개 부분, 검증 결과
- **특징**: 증명에 포함되지 않음

Auxiliary Input (보조 입력)

- **정의**: 증명자만 알고 있는 비밀 값들
- **용도**: 계산 과정의 중간 결과, 비밀 증인
- **특징**: 영지식성 보장 대상

상수 변수

- **정의**: 항상 1의 값을 가지는 특수 변수 (인덱스 0)
- **용도**: 상수항 표현, 제약 조건 정규화

변수 할당 클래스

기본 구조

```
template<typename FieldT>
class VariableAssignment {
private:
vector<FieldT> values;
size_t num_inputs;
size_t num_aux;
public:
VariableAssignment(size_t num_vars, size_t inputs, size_t aux)
: values(num_vars), num_inputs(inputs), num_aux(aux) {
// 상수 변수 (인덱스 0) 초기화
if (num_vars > 0) {
values[0] = FieldT::one();
}
}
// 값 설정
void set_value(size_t index, const FieldT& value) {
if (index == 0) {
throw invalid_argument("Cannot modify constant variable");
}
if (index < values.size()) {
values[index] = value;
```

```
}

}

// 값 조회

FieldT get_value(size_t  index) const {

if (index < values.size()) {

return  values[index];

}

return FieldT::zero();

}

// 전체 할당 반환

const vector<FieldT>&  get_assignment() const {

return values;

}

// 공개 입력만 추출

vector<FieldT> get_primary_input() const {

vector<FieldT> primary(num_inputs);

for (size_t i = 0; i < num_inputs; i++) {

primary[i] = values[i + 1]; // 인덱스 0은 상수

}

return primary;

}

// 보조 입력만 추출

vector<FieldT> get_auxiliary_input() const {
```

```
vector<FieldT> auxiliary(num_aux);

size_t aux_start = 1 + num_inputs;

for (size_t i = 0; i < num_aux; i++) {

auxiliary[i] = values[aux_start + i];

}

return auxiliary;

}

};
```

증인 생성 과정

증인 생성기 인터페이스

```
template<typename FieldT>

class WitnessGenerator {

public:

virtual  ~WitnessGenerator() = default;

// 공개 입력으로부터 전체 증인 생성

virtual VariableAssignment<FieldT> generate_witness(

const vector<FieldT>&  primary_input

) = 0;

// 부분 증인 생성 (점진적 구성)

virtual  void  generate_partial_witness(
```

```
VariableAssignment<FieldT>&  assignment,

size_t  start_constraint,

size_t  end_constraint

) = 0;

};
```

산술 회로 증인 생성기

```
template<typename FieldT>

class ArithmeticCircuitWitnessGenerator : public WitnessGenerator<FieldT> {

private:

struct Gate {

enum Type { ADD, MUL, CONST, INPUT };

Type type;

size_t left_input;

size_t right_input;

size_t output;

FieldT constant_value;

};

vector<Gate> gates;

vector<size_t> input_variables;

vector<size_t> output_variables;

public:
```

```
// 게이트 추가
void add_addition_gate(size_t left, size_t right, size_t output) {
gates.push_back({Gate::ADD, left, right, output, FieldT::zero()});
}
void add_multiplication_gate(size_t left, size_t right, size_t output) {
gates.push_back({Gate::MUL, left, right, output, FieldT::zero()});
}
void add_constant_gate(size_t output, const FieldT& value) {
gates.push_back({Gate::CONST, 0, 0, output, value});
}
// 증인 생성 구현
VariableAssignment<FieldT> generate_witness(
const vector<FieldT>& primary_input
) override {
size_t total_vars = compute_total_variables();
VariableAssignment<FieldT> assignment(
total_vars,
primary_input.size(),
total_vars - primary_input.size() - 1
);
// 공개 입력 설정
for (size_t i = 0; i < primary_input.size(); i++) {
assignment.set_value(input_variables[i], primary_input[i]);
```

```
}

// 게이트별 계산 수행

for (const  auto& gate : gates) {

execute_gate(gate, assignment);

}

return assignment;

}

private:

void  execute_gate(

const Gate&  gate,

VariableAssignment<FieldT>&  assignment

) {

switch (gate.type) {

case Gate::ADD: {

FieldT left_val = assignment.get_value(gate.left_input);

FieldT right_val = assignment.get_value(gate.right_input);

assignment.set_value(gate.output, left_val + right_val);

break;

}

case Gate::MUL: {

FieldT left_val = assignment.get_value(gate.left_input);

FieldT right_val = assignment.get_value(gate.right_input);

assignment.set_value(gate.output, left_val * right_val);
```

```
break;

}

case Gate::CONST: {

assignment.set_value(gate.output, gate.constant_value);

break;

}

case Gate::INPUT: {

// 입력은 이미 설정됨

break;

}

}

}

};
```

변수 인덱싱 전략

순차적 인덱싱

```
template<typename FieldT>

class SequentialIndexer {

private:

size_t next_index;

map<string, size_t> variable_map;
```

```
public:
SequentialIndexer() : next_index(1) { // 0은 상수용 예약
variable_map["ONE"] = 0;
}
size_t allocate_variable(const string& name = "") {
size_t index = next_index++;
if (!name.empty()) {
variable_map[name] = index;
}
return index;
}
size_t get_variable_index(const string& name) const {
auto it = variable_map.find(name);
if (it != variable_map.end()) {
return it->second;
}
throw invalid_argument("Variable not found: " + name);
}
};
```

계층적 인덱싱

```
template<typename FieldT>
```

```
class HierarchicalIndexer {

private:

struct Scope {

string name;

size_t start_index;

size_t end_index;

map<string, size_t> local_variables;

};

vector<Scope> scope_stack;

size_t global_next_index;

public:

HierarchicalIndexer() : global_next_index(1) {

// 전역 스코프 생성

push_scope("global");

}

void  push_scope(const string&  scope_name) {

Scope new_scope;

new_scope.name = scope_name;

new_scope.start_index = global_next_index;

new_scope.end_index = global_next_index;

scope_stack.push_back(new_scope);

}

void  pop_scope() {
```

```
if (scope_stack.size() > 1) { // 전역 스코프는 유지
scope_stack.back().end_index = global_next_index;
scope_stack.pop_back();
}
}
size_t  allocate_variable(const string&  name) {
size_t index = global_next_index++;
scope_stack.back().local_variables[name] = index;
return index;
}
size_t  resolve_variable(const string&  name) const {
// 현재 스코프부터 역순으로 검색
for (auto it = scope_stack.rbegin(); it != scope_stack.rend(); ++it) {
auto var_it = it->local_variables.find(name);
if (var_it != it->local_variables.end()) {
return  var_it->second;
}
}
throw  invalid_argument("Variable not found: " + name);
}
};
```

증인 검증과 최적화

증인 검증기

```
template<typename FieldT>
class WitnessValidator {
public:
struct ValidationResult {
bool is_valid;
vector<string> errors;
vector<size_t> unassigned_variables;
};
static ValidationResult validate_witness(
const R1CSConstraintSystem<FieldT>&  cs,
const VariableAssignment<FieldT>&  assignment
) {
ValidationResult result;
result.is_valid = true;
// 모든 변수가 할당되었는지 확인
for (size_t i = 0; i < cs.num_vars(); i++) {
if (assignment.get_value(i) == FieldT::zero() && i != 0) {
// 0이 유효한 값일 수도 있으므로 추가 검사 필요
result.unassigned_variables.push_back(i);
}
}
```

```
// 제약 조건 만족 여부 확인

auto verification_result = R1CSVerifier<FieldT>::verify_system(

cs, assignment.get_assignment()

);

if (!verification_result.is_satisfied) {

result.is_valid = false;

result.errors.push_back(verification_result.error_message);

}

return result;

}

};
```

증인 압축

```
template<typename FieldT>

class WitnessCompressor {

public:

// 희소 증인 표현

struct SparseWitness {

vector<size_t> indices;

vector<FieldT> values;

size_t total_size;

};
```

```
static SparseWitness compress_witness(
const VariableAssignment<FieldT>&  assignment
) {
SparseWitness compressed;
const  auto& values = assignment.get_assignment();
compressed.total_size = values.size();
for (size_t i = 0; i < values.size(); i++) {
if (values[i] != FieldT::zero()) {
compressed.indices.push_back(i);
compressed.values.push_back(values[i]);
}
}
return compressed;
}
static VariableAssignment<FieldT> decompress_witness(
const SparseWitness&  compressed,
size_t  num_inputs,
size_t  num_aux
) {
VariableAssignment<FieldT> assignment(
compressed.total_size,
num_inputs,
num_aux
```

```
);

for (size_t i = 0; i < compressed.indices.size(); i++) {

size_t index = compressed.indices[i];

FieldT value = compressed.values[i];

if (index != 0) { // 상수 변수는 자동 설정됨

assignment.set_value(index, value);

}

}

return assignment;

}

};
```

병렬 증인 생성

병렬 처리 전략

```
template<typename FieldT>

class ParallelWitnessGenerator {

private:

struct ComputationTask {

size_t output_variable;

function<FieldT(const VariableAssignment<FieldT>&)> computation;

vector<size_t> dependencies;
```

```
};

vector<ComputationTask> tasks;

public:

void  add_computation_task(

size_t  output_var,

function<FieldT(const VariableAssignment<FieldT>&)> compute_func,

const vector<size_t>&  deps

) {

tasks.push_back({output_var, compute_func, deps});

}

VariableAssignment<FieldT> generate_witness_parallel(

const vector<FieldT>&  primary_input,

size_t  num_vars,

size_t  num_aux

) {

VariableAssignment<FieldT> assignment(

num_vars,

primary_input.size(),

num_aux

);

// 공개 입력 설정

for (size_t i = 0; i < primary_input.size(); i++) {

assignment.set_value(i + 1, primary_input[i]);
```

```
}

// 의존성 그래프 기반 위상 정렬

vector<vector<size_t>> levels = topological_sort_by_levels();

// 레벨별 병렬 실행

for (const  auto& level : levels) {

#pragma  omp  parallel  for

for (size_t i = 0; i < level.size(); i++) {

size_t task_index = level[i];

const  auto& task = tasks[task_index];

FieldT result = task.computation(assignment);

assignment.set_value(task.output_variable, result);

}

}

return assignment;

}

private:

vector<vector<size_t>> topological_sort_by_levels() {

// 위상 정렬 구현

vector<vector<size_t>> levels;

vector<int> in_degree(tasks.size(), 0);

// 진입 차수 계산

for (size_t i = 0; i < tasks.size(); i++) {

for (size_t dep : tasks[i].dependencies) {
```

```
in_degree[i]++;

}

}

// 레벨별 분류

queue<size_t> ready_tasks;

for (size_t i = 0; i < tasks.size(); i++) {

if (in_degree[i] == 0) {

ready_tasks.push(i);

}

}

while (!ready_tasks.empty()) {

vector<size_t> current_level;

size_t level_size = ready_tasks.size();

for (size_t i = 0; i < level_size; i++) {

size_t task_id = ready_tasks.front();

ready_tasks.pop();

current_level.push_back(task_id);

// 의존성 업데이트

for (size_t j = 0; j < tasks.size(); j++) {

for (size_t dep : tasks[j].dependencies) {

if (dep == task_id) {

in_degree[j]--;

if (in_degree[j] == 0) {
```

```
ready_tasks.push(j);

}

}

}

}

}

levels.push_back(current_level);

}

return levels;

}

};
```

다음 단계

이제 R1CS 관계 시스템을 마쳤으므로, 프로토콜 분석 섹션에서 실제 zkSNARK 프로토콜들의 구현을 살펴보겠습니다.

← 이전: 제약 시스템 | 다음: 프로토콜 분석 →